

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA1220	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	ASWIN M	Reg. No.:	192412437
Section / Batch:	B.E (ECE) 3 rd Year	Date:	10/08/2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply techniques such as pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.
2. Develop a computer system model using a hierarchical bus structure to transfer data among the CPU, memory, and I/O devices, and analyse how bus priority, bandwidth, and transfer conflicts affect overall system performance.
3. Develop a processor model that supports multiple instruction issue, out-of-order execution, and speculative execution, and analyse how instruction scheduling, execution units, and simultaneous multithreading affect instruction throughput and processor resource utilization.
4. Design and simulate a processor data path that performs register-to-register data transfers and arithmetic operations such as increment, decrement, addition, and subtraction, and analyse the control signals and execution sequence involved in each operation.
5. Simulate a pipelined processor executing a sequence of arithmetic, memory, and branch instructions, and calculate the pipeline execution time, throughput, and speedup by comparing the results with sequential instruction execution.

Code of Conduct

I Aswin M - 192412437 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

Question 1

Name : Aswin M

Reg No: 192412437

Subject Code: CSA1220

Pipeline Hazard Management and Performance Optimization

Objective

To construct a pipeline scenario containing dependent instructions, a branch instruction, and competing resource requirements, and to apply pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.

Pipeline Structure

A 5-stage instruction pipeline is considered:

IF – Instruction Fetch

ID – Instruction Decode

EX – Execute

MEM – Memory Access

WB – Write Back

Instruction Sequence

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4

I3: SUB R5, R3, R6

I4: BEQ R5, R0, TARGET

I5: ADD R7, R8, R9

I6: LOAD R10, 0(R11)

TARGET:

I7: ADD R12, R10, R13

Hazards Identified

1. Data Hazard:

I2 depends on the result of I1, since I2 requires the value stored in R1. A load-use hazard occurs, so one pipeline stall is required.

2. Data Dependency:

I3 depends on the result of I2. Operand forwarding is used to directly transfer the result to the EX stage without waiting for register write-back.

3. Control Hazard:

I4 is a branch instruction. Branch prediction is used to predict whether the branch is taken. If the prediction is

correct, pipeline execution continues without a major penalty. If incorrect, the wrongly fetched instructions are flushed.

4. Structural Hazard:

If instruction fetch and data access use the same memory, both operations may compete for the memory resource. Resource duplication solves this by using separate instruction and data memories.

Pipeline Execution

Cycle 1 2 3 4 5 6 7

I1 IF ID EX MEM WB

I2 IF ID ST EX MEM WB

I3 IF ID EX MEM WB

Here, **ST** represents one stall cycle caused by the load-use dependency.

Performance Calculation

For 7 instructions without pipelining:

$$T_{\text{sequential}} = 7 \times 5 = 35 \text{ cycles}$$

For a 5-stage pipeline:

$$T_{\text{pipeline}} = 5 + (7 - 1) = 11 \text{ cycles}$$

Including one stall cycle:

$$T_{\text{pipeline}} = 11 + 1 = 12 \text{ cycles}$$

Therefore, the speedup is:

$$\text{Speedup} = \frac{35}{12} = 2.92$$

Result

The proposed pipeline reduces the execution time from 35 cycles to 12 cycles, providing approximately 2.92× speedup. Operand forwarding reduces data-hazard stalls, branch prediction reduces branch penalties, and resource duplication eliminates structural conflicts. Thus, these techniques improve pipeline efficiency and overall processor performance.

Question 2

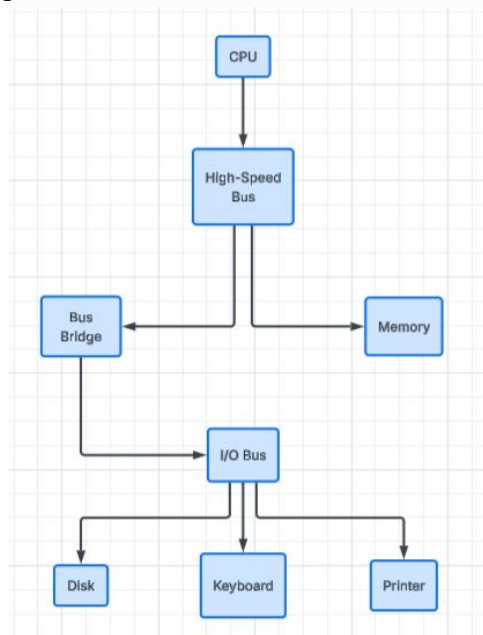
Hierarchical Bus Organization and System Performance Analysis

Objective

To develop a computer system model using a hierarchical bus structure for transferring data among the CPU, memory, and I/O devices, and to analyze how bus priority, bandwidth, and transfer conflicts affect overall system performance.

Pipeline Structure

A hierarchical bus organization is considered with a high-speed system bus connecting the CPU and memory, and a slower I/O bus connecting peripheral devices.



Bus Organization

The system consists of:

CPU: Generates memory and I/O requests.

Memory: Stores instructions and data.

System Bus: Provides high-speed communication between CPU and memory.

Bus Bridge: Connects the high-speed system bus to the slower I/O bus.

I/O Bus: Connects peripheral devices to the system.

Bus Priority

When multiple devices request the bus simultaneously, a **priority-based bus arbitration** mechanism is used.

The priority is assumed as:

CPU/Memory → High Priority

Disk I/O → Medium Priority

Keyboard → Low Priority

Printer → Low Priority

The device with higher priority receives access to the bus first. This prevents important CPU and memory transfers from being delayed by slower I/O devices.

Bus Bandwidth

Assume:

System bus bandwidth = **1 GB/s**

I/O bus bandwidth = **250 MB/s**

Data transfer requested by CPU = **400 MB**

Data transfer requested by I/O device = **100 MB**

CPU transfer time:

$$T_{\text{CPU}} = \frac{400 \text{ MB}}{1 \text{ GB/s}} = 0.4 \text{ s}$$

I/O transfer time:

$$T_{\text{I/O}} = \frac{100 \text{ MB}}{250 \text{ MB/s}} = 0.4 \text{ s}$$

Therefore, higher bus bandwidth allows the CPU and memory to transfer large amounts of data efficiently.

Transfer Conflict

A transfer conflict occurs when multiple devices request the same bus at the same time.

For example:

CPU → Memory

Disk → I/O Bus

Printer → I/O Bus

If two devices request the I/O bus simultaneously, only one device can use the shared bus at a time. The remaining device must wait.

This produces:

Increased waiting time

Reduced bus utilization

Increased data transfer latency

Lower overall system performance

Effect of Hierarchical Bus

The hierarchical structure reduces unnecessary traffic on the high-speed system bus.

I/O traffic is handled by the I/O bus, while CPU-memory communication continues through the high-speed system bus.

Performance Analysis

If the system bus is shared by CPU, memory, and all I/O devices, frequent I/O requests can create bus contention and increase CPU waiting time.

With a hierarchical bus:

CPU-memory transfers receive higher priority.

I/O traffic is isolated to the I/O bus.

Bus conflicts are reduced.

High-speed devices can use the system bus efficiently.

Overall system throughput is improved.

Result

The hierarchical bus structure provides efficient communication between the **CPU, memory, and I/O devices**. Bus priority ensures that critical transfers are serviced first, while appropriate bandwidth reduces transfer time. Separating I/O traffic from the main system bus minimizes transfer conflicts and improves overall system performance.

Question 3

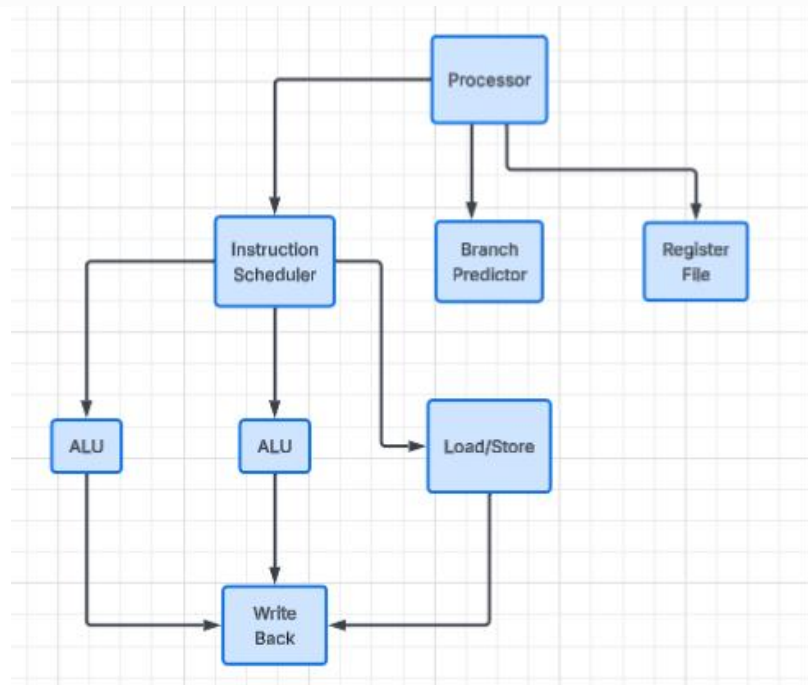
Superscalar Processor with Out of Order and Speculative Execution

Objective

To develop a processor model that supports multiple instruction issue, out-of-order execution, and speculative execution, and to analyze how instruction scheduling, execution units, and simultaneous multithreading affect instruction throughput and processor resource utilization.

Processor Structure

A **superscalar processor** with multiple execution units is considered. The processor can issue more than one instruction per clock cycle.



Multiple Instruction Issue

The processor can issue **two instructions per clock cycle** when there are no dependencies.

Cycle 1 → I1, I2

Cycle 2 → I3, I4

Cycle 3 → I5, I6

However, I3 depends on I1, I4 depends on I2, and I6 depends on I5. Therefore, instruction scheduling is required to avoid unnecessary waiting.

Out-of-Order Execution

Instructions are executed when their required operands and execution units are available rather than strictly following program order.

For example:

I1: ADD R1,R2,R3

I2: SUB R4,R5,R6

I3: MUL R7,R1,R8

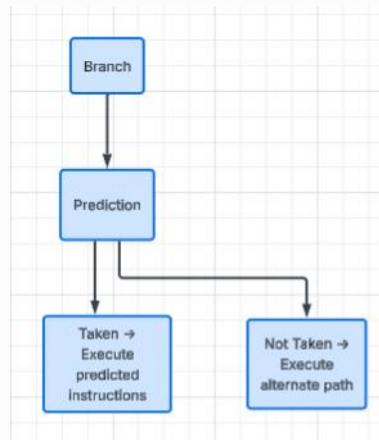
I4: ADD R9,R4,R10

I3 must wait for I1, while I4 can execute after I2 is completed. Independent instructions can therefore use available execution units without waiting for unrelated instructions.

This improves processor utilization and reduces idle execution units.

Speculative Execution

The processor predicts the outcome of branch instructions and executes instructions from the predicted path before the branch result is confirmed.



If the prediction is correct, execution continues without a major delay. If the prediction is incorrect, speculative instructions are discarded and the correct instructions are fetched.

Instruction Scheduling

The instruction scheduler identifies independent instructions and assigns them to available execution units.

Simultaneous Multithreading

Simultaneous multithreading (SMT) allows instructions from multiple threads to use processor resources at the same time.

When one thread is waiting for memory or another dependency, instructions from another thread can use the available execution units.

Performance Analysis

Assume:

Single-issue processor = 1 instruction/cycle

Superscalar processor = 2 instructions/cycle

Total instructions = 6

Ideal single-issue execution:

$$T_{\text{single}} = 6 \text{ cycles}$$

Ideal dual-issue execution:

$$T_{\text{dual}} = 26 = 3 \text{ cycles}$$

Therefore, ideal speedup is:

$$\text{Speedup} = \frac{36}{26} = 2 \times \text{Speedup} = 2 \times$$

In practice, dependencies, branch mispredictions, and resource conflicts reduce the actual speedup.

Result

The proposed superscalar processor improves instruction throughput by using multiple instruction issue, out-of-order execution, speculative execution, instruction scheduling, and simultaneous multithreading. These techniques allow available execution units to remain busy and reduce processor idle time. The ideal dual-issue processor can achieve up to $2\times$ instruction throughput compared with a single-issue processor.

Question 4

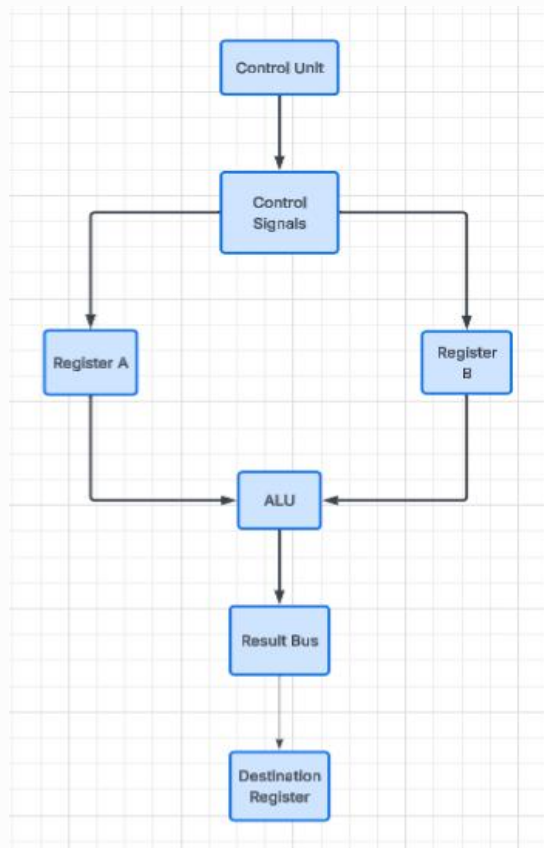
Processor Datapath for Register Transfers and Arithmetic Operations

Objective

To design and simulate a processor datapath that performs register to register data transfers and arithmetic operations such as increment, decrement, addition, and subtraction, and to analyze the control signals and execution sequence involved in each operation.

Processor Datapath

The processor consists of a register file, ALU, multiplexer, and control unit. The control unit generates the required control signals for selecting registers and ALU operations.



Register Transfer

For a register to register transfer:

$$R2 \leftarrow R1$$

The contents of $R1$ are transferred to $R2$.

Arithmetic Operations

The ALU performs four basic operations.

1. Addition

Operation:

$$R3 \leftarrow R1 + R2$$

Control signals:

R1out $\rightarrow$ ALU input A

R2out $\rightarrow$ ALU input B

ALU = ADD

R3in $\rightarrow$ Store result

2. Subtraction

Operation:

$$R3 \leftarrow R1 - R2$$

Control signals:

R1out $\rightarrow$ ALU input A

R2out $\rightarrow$ ALU input B

ALU = SUB

R3in $\rightarrow$ Store result

3. Increment

Operation:

$$R1 \leftarrow R1 + 1$$

Control signals:

R1out $\rightarrow$ ALU input

ALU = INC

R1in $\rightarrow$ Store result

4. Decrement

Operation:

$$R1 \leftarrow R1 - 1$$

Control signals:

R1out $\rightarrow$ ALU input

ALU = DEC

R1in $\rightarrow$ Store result

Control Signal Table

Operation	Source	ALU Operation	Destination
Transfer	R1	PASS	R2
Addition	R1, R2	ADD	R3
Subtraction	R1, R2	SUB	R3
Increment	R1	INC	R1
Decrement	R1	DEC	R1

Result

The designed processor datapath successfully performs register transfers, addition, subtraction, increment, and decrement using the ALU and control unit. The control signals determine the source register, ALU operation, and destination register. The datapath demonstrates how data moves between registers through the bus and ALU during each processor operation.

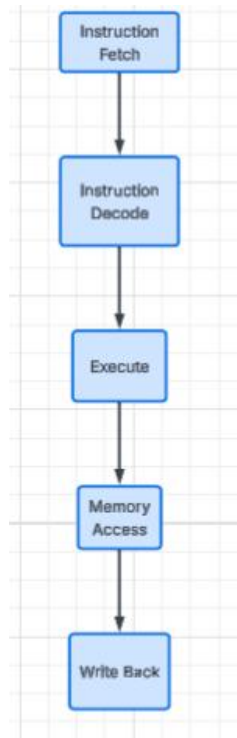
Question 5

Pipelined Processor Simulation and Performance Analysis

Objective

To simulate a pipelined processor executing a sequence of arithmetic, memory, and branch instructions, and to calculate the pipeline execution time, throughput, and speedup by comparing the results with sequential instruction execution.

Pipeline Structure



Instruction Sequence

The following instructions are used for the simulation:

I1: ADD R1, R2, R3

I2: LOAD R4, 0(R5)

I3: SUB R6, R1, R4

I4: BEQ R6, R0, TARGET

I5: ADD R7, R8, R9

The instruction sequence contains:

Arithmetic instructions: ADD, SUB

Memory instruction: LOAD

Branch instruction: BEQ

Pipeline Execution

The instructions pass through the five pipeline stages as follows:

Cycle	1	2	3	4	5	6	7	8	
I1 ADD	IF	ID	EX	MEM	WB				
I2 LOAD		IF	ID	EX	MEM	WB			
I3 SUB			IF	ID	EX	MEM	WB		
I4 BEQ				IF	ID	EX	MEM	WB	
I5 ADD					IF	ID	EX	MEM	WB

After the pipeline is filled, multiple instructions are processed simultaneously.

Sequential Execution

Without pipelining, each instruction completes all five stages before the next instruction starts.

For 5 instructions:

$$T_{\text{sequential}} = 5 \times T_{\text{sequential}} = 25 \text{ cycles}$$

Pipeline Execution Time

For a 5 stage pipeline and 5 instructions:

$$T_{\text{pipeline}} = k + (n - 1)$$

Where:

$$k = 5, n = 5$$

Therefore:

$$T_{\text{pipeline}} = 5 + (5 - 1) T_{\text{pipeline}} = 9 \text{ cycles}$$

Speedup

Speedup is calculated using:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{pipeline}}} \quad \text{Speedup} = \frac{25}{9} \quad \text{Speedup} = 2.78$$

Thus, the pipelined processor provides approximately 2.78 times speedup compared with sequential execution.

Throughput

Throughput is the number of instructions completed per cycle.

For the pipelined processor:

Throughput=95 Throughput=0.556 instructions/cycle
--

After the pipeline becomes full, the ideal throughput approaches:

1 instruction/cycle

Performance Comparison

Parameter	Sequential	Pipelined
Number of instructions	5	5
Pipeline stages	5	5
Execution time	25 cycles	9 cycles
Throughput	0.2 instructions/cycle	0.556 instructions/cycle

Result

The pipelined processor completes the five instructions in 9 cycles, compared with 25 cycles for sequential execution. The resulting speedup is approximately 2.78×, and the throughput improves from 0.2 instructions/cycle to 0.556 instructions/cycle. Therefore, pipelining significantly improves processor performance by allowing multiple instructions to be processed simultaneously.